Dodona 02 — metrics

what the numbers mean 2026-10-11 02:12 UTC

Objective

What does each metric mean, what does good look like, and what can it NOT tell you?

Falsified by: a metric in the catalogue that no longer exists, or a trap that no longer reproduces

0. Traffic for this run

	count
requests sent	12

Refusals here are gateway decisions, not GPU limits.

PASS some traffic completed

>> 12 requests, 0 refused. Every number below covers a 15-minute window that includes this traffic.

Is the GPU busy?

metric	now	answers	means
vllm:kv_cache_usage_perc	0	How full is the KV cache?	fraction of a pod's own KV in use — the signal
vllm:num_requests_running	0	How many sequences are resident?	requests the scheduler is stepping right now
vllm:num_requests_waiting	0	Is anything queued?	admitted but not yet running — the scheduler i
vllm:num_preemptions_total	0	Did we overcommit?	the engine evicted a running sequence to fit a

vllm:kv_cache_usage_perc — CANNOT TELL YOU: it is a RATIO of that slice. 50% of 4.1 GiB is not 50% of the card.

vllm:num_requests_running — CANNOT TELL YOU: says nothing about queued work, which is where latency comes from.

vllm:num_requests_waiting — CANNOT TELL YOU: zero can mean idle OR that admission is shedding before the queue.

vllm:num_preemptions_total — CANNOT TELL YOU: a lagging counter: by the time it moves, a user already stalled.

How fast is it?

metric	now	answers	means
vllm:time_to_first_token_seconds	0.088	Dead air before the answer starts	TTFT p95 — for a voice agent th
vllm:time_per_output_token_seconds	no data	How it reads aloud	ITL — inter-token latency, the
vllm:e2e_request_latency_seconds	0.285	The whole request	E2E p95 — what the caller waits
vllm:request_queue_time_seconds	0.285	Time lost before the GPU touched it	queue p95 — the part admission

vllm:time_to_first_token_seconds — CANNOT TELL YOU: a p95 over a 5m window hides a 30s stall that hit three users.

vllm:time_per_output_token_seconds — CANNOT TELL YOU: an average ITL can look fine while the gaps are bunched.

vllm:e2e_request_latency_seconds — CANNOT TELL YOU: includes queue time, so it moves when load changes, not just the

vllm:request_queue_time_seconds — CANNOT TELL YOU: high queue with low KV means the SCHEDULER is binding, not m

Did the KV hop move bytes?

metric	now	answers
vllm:external_prefix_cache_queries_total{pod=~"vllm-decode.*"}	1.398e+04	Was the connec
vllm:external_prefix_cache_hits_total{pod=~"vllm-decode.*"}	1.398e+04	Did it find bl
vllm:prompt_tokens_by_source_total{source="external_kv_transfer",pod=~"vllm-decode.*"}	1.398e+04	Did bytes actu

orch_kv_transfer_total	63	What did the r

vllm:external_prefix_cache_queries_total{pod=~"vllm-decode.*"} — CANNOT TELL YOU: being asked is not being answere
vllm:external_prefix_cache_hits_total{pod=~"vllm-decode.*"} — CANNOT TELL YOU: A LOOKUP, NOT A TRANSFER. The pul
vllm:prompt_tokens_by_source_total{source="external_kv_transfer",pod=~"vllm-decode.*"} — CANNOT TELL YOU: the ho
vllm:prompt_tokens_by_source_total{source="local_compute",pod=~"vllm-decode.*"} — CANNOT TELL YOU: nothing — th
vllm:prompt_tokens_by_source_total{source="local_cache_hit",pod=~"vllm-decode.*"} — CANNOT TELL YOU: ONE WORD
orch_kv_transfer_total — CANNOT TELL YOU: bookkeeping. It counts intent, not bytes, and will happily report throughput fo

Is admission doing anything?

metric	now	answers	means
orch_requests_total	63	How much arrived?	requests past guardrails
orch_completed_total	63	How much was served?	completions
orch_shed_total	0	How much was refused?	shed, by why
orch_overflow_total	0	Did anything leave?	bursted to an external provider

orch_shed_total — CANNOT TELL YOU: a 429 is the tenant over quota; a 503 is capacity. Reporting them together makes a
orch_overflow_total — CANNOT TELL YOU: capped for the process lifetime, so 0 can mean 'never needed' or 'budget alread

Is routing choosing well?

metric	now	answers	means
orch_pick_total	126	How many placements?	a pod was chosen
orch_sticky_total	0	How many were prefix-warm?	placed where the prefix already lives
orch_pick_unknown_snapshot_total	0	How many were blind?	chosen with no usable signal

orch_sticky_total — CANNOT TELL YOU: only meaningful against the total — 10 sticky of 10 is perfect, of 1000 is noise.
orch_pick_unknown_snapshot_total — CANNOT TELL YOU: high here means the scorers are inert and placement is effective

Is the autoscaler awake?

metric	now	answers	means
keda_scaler_metrics_value	0	What does KEDA see?	the value its trigger reads
kube_deployment_status_replicas{deployment="vllm-decode"}	1	What has it done?	actual replicas

keda_scaler_metrics_value — CANNOT TELL YOU: it polls — it lags the real signal.
kube_deployment_status_replicas{deployment="vllm-decode"} — CANNOT TELL YOU: scaling UP takes minutes on a pod t

2. The SLAs

promise	target	why that number
TTFT p95	1.5s	A voice agent's latency is dead air. Past about 1.5s the listener starts talkin
turn p95	30s	A research turn runs tool calls and reads sources. 30s is the budget for the wh
preemptions	0	vLLM preempting a sequence is a stall mid-sentence in a spoken answer. This is
shed rate at target load	0.05	503s are allowed but rare. Shedding early beats preempting; a system shedding 3

Concurrency is deliberately absent: it is derived in 03-capacity from a measured saturation curve. A concurrency target pic

>> Each has a metric that proves it: TTFT and turn from the latency histograms, preemptions

from vllm:num_preemptions_total, shed rate from orch_shed_total over orch_requests_total.

3. Where the time goes

stage	mean	of end-to-end	what happens here
waiting to be scheduled	23.2 ms	32%	admitted, not yet running
prefill	29.4 ms	40%	the prompt becomes KV
decode	4.8 ms	7%	tokens come out
end to end	72.7 ms	100%	over 126 requests

time to first token	seconds
p50	0.049
p90	0.076
p99	0.098
mean	0.068

MET TTFT p95 0.088s <= 1.5s

>> A request takes 73 ms end to end. Prefill is the largest share at 40%, waiting is 32%, and decode only 7% — these answers are a few tokens long, so almost all the work is reading the prompt rather than writing the reply. The three phases account for 79% of the total; the rest is scheduling overhead. First token lands at 76 ms p90, well inside the 1.5s promise.

4. Component checks

prefill / decode roles

metric	value	what it means
vllm:prompt_tokens_total{pod=~"vllm-prefill.*"}	7.003e+04	prefill should carry prompt work
vllm:generation_tokens_total{pod=~"vllm-prefill.*"}	63	near 0: its leg is sent with max_tokens=1
vllm:generation_tokens_total{pod=~"vllm-decode.*"}	126	decode does the generating

scheduler

metric	value	what it means
vllm:num_requests_running	0	in flight on the GPU
vllm:num_requests_waiting	0	queued behind max-num-seqs
vllm:num_preemptions_total	0	must be 0 — a preempt is a stall mid-answer
vllm:kv_cache_usage_perc	0	the signal admission reads

KV hop

metric	value	what it means
vllm:external_prefix_cache_queries_total{pod=~"vllm-decode.*"}	1.398e+04	decode looked
vllm:external_prefix_cache_hits_total{pod=~"vllm-decode.*"}	1.398e+04	blocks found r
vllm:prompt_tokens_by_source_total{source="external_kv_transfer",pod=~"vllm-decode.*"}	1.398e+04	KV that arrive

vllm:prompt_tokens_by_source_total{source="local_compute",pod=~"vllm-decode.*"}	0	prompt work de
orch_kv_transfer_total	63	router bookkee

admission

metric	value	what it means
orch_requests_total	63	reached the gateway
orch_completed_total	63	served
orch_shed_total	0	refused, by reason below
orch_overflow_total	0	bursted to a provider

routing

metric	value	what it means
orch_pick_total	126	a pod was chosen
orch_sticky_total	0	placed on a prefix-warm pod
orch_pick_unknown_snapshot_total	0	placed blind — the scorer had no signal

KEDA

metric	value	what it means
keda_scaler_metrics_value	0	what KEDA sees
kube_deployment_status_replicas{deployment="vllm-decode"}	1	what it has done about it

4.1 The KV connector

stage	tokens	
asked	13,985	decode looked for blocks on the producer
found	13,985	the lookup matched
arrived	13,985	KV that came over the wire

PASS the connector has delivered KV
PASS found and arrived agree — lookups are completing as transfers
PASS decode recomputes 0.0% of its prompt tokens

>> Of 70,033 prompt tokens decode needed, 13,985 (20.0%) arrived as KV from prefill, 56,048 came from its own prefix cache, and it computed 0. Decode does no prompt work at all — the hop is carrying the whole prompt, which is exactly what disaggregation is for.

4.2 Prefill and decode

	last 1h	
prefill requests	36	its leg of each split
prefill generated	36	1.00 tokens per request
decode generated	72	the actual answers

Both from the same window and the same pods, so the ratio is real.

```
PASS  prefill emits 1.00 tokens per request (max_tokens=1 means ~1.0)
```

>> Over the last 1h prefill answered 36 requests with 36 tokens — 1.00 each, which is the max_tokens=1 leg doing prompt work only, against decode's 72. The two pools are doing different jobs.

4.3 Slice capacity

	KV available	full-length sequences @16k
the whole H100	~64 GiB	~73.5
one 12 GiB HAMi slice	4.10 GiB	4.7

CUDA-graph capture scales with available memory, so the whole-card figure cannot be divided down — it is measured per

kv_cache_usage_perc	ratio	of this slice
now	0.0%	0.00 GiB
peak over 1h	0.0%	0.00 GiB

```
PASS  KV peaked at 0.0% — below the 0.90 admission threshold
```

>> A pod holds 4.10 GiB of KV, which is 4.7 full-length sequences — that is the denominator for every concurrency claim, and it is 15x smaller than the whole card would suggest. Over the last hour KV peaked at 0.0% (0.00 GiB), so a kv_threshold policy had nothing to decide. Making it bind is what 03-capacity is for.

No panels rendered — set GRAFANA_PASSWORD (printed by make observability) and check renderer.yaml is deployed.

vllm

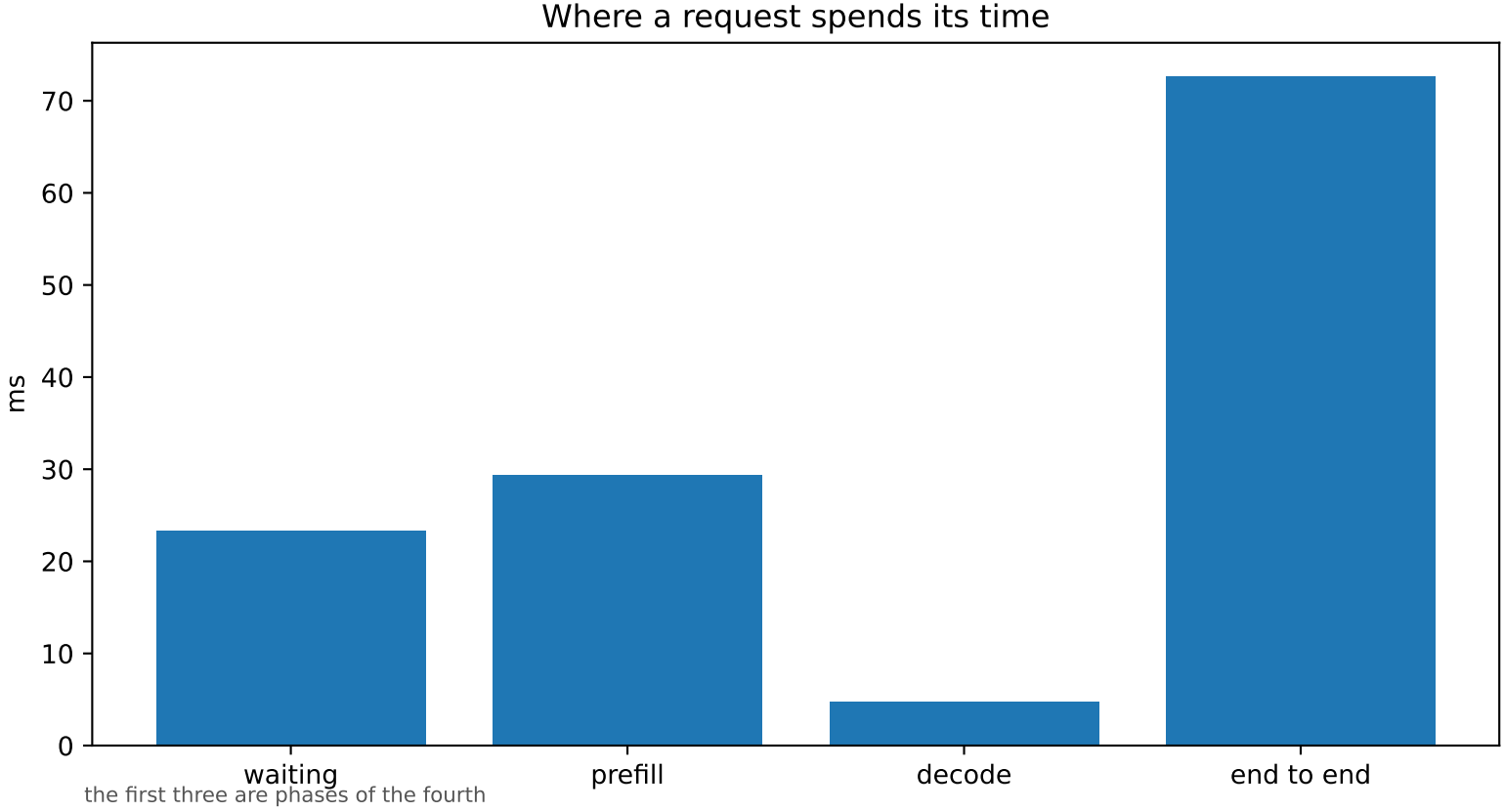
```
http://127.0.0.1:31495/d/dodona-vllm?from=now-30m&to=now
```

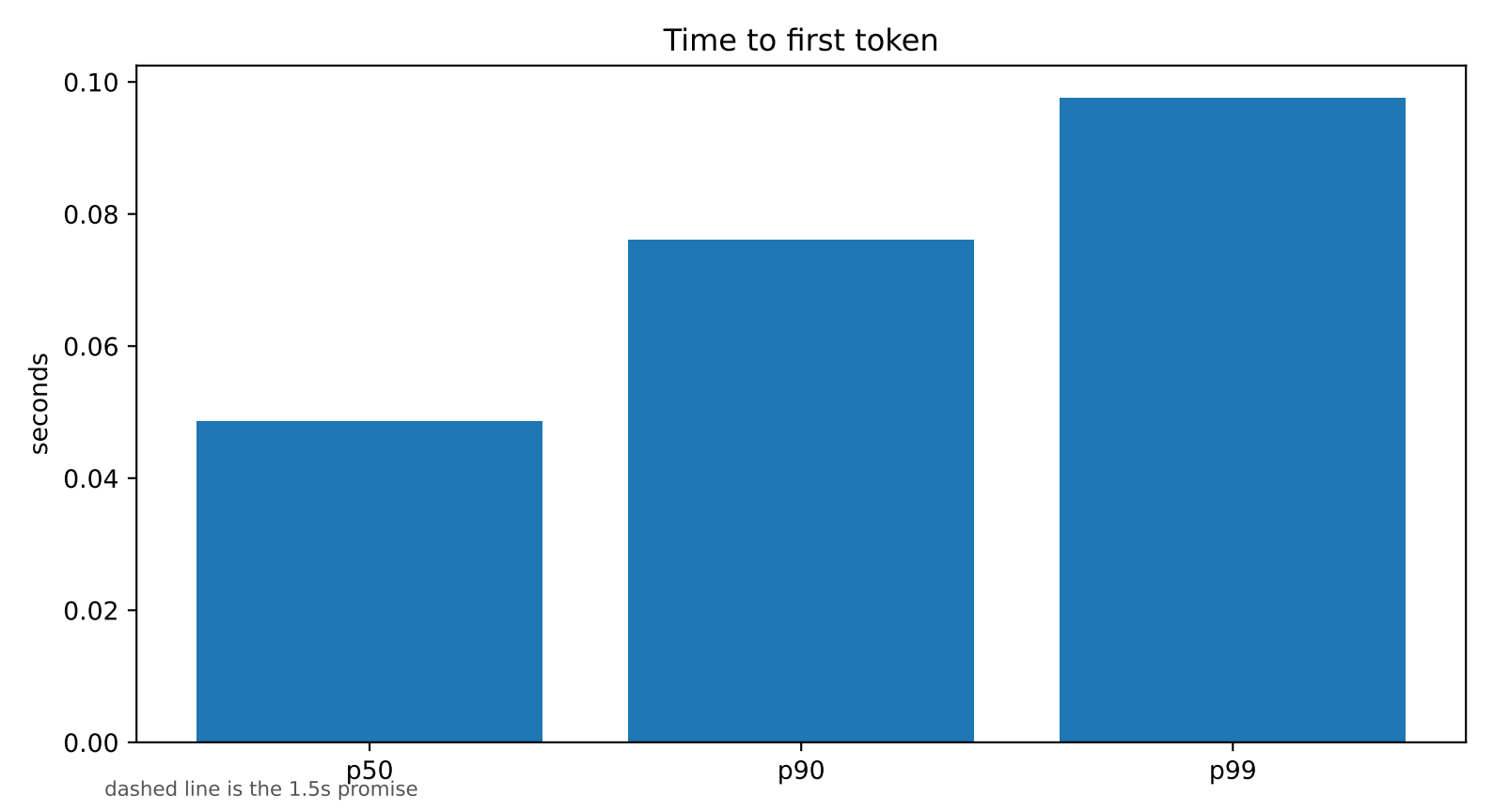
mooncake

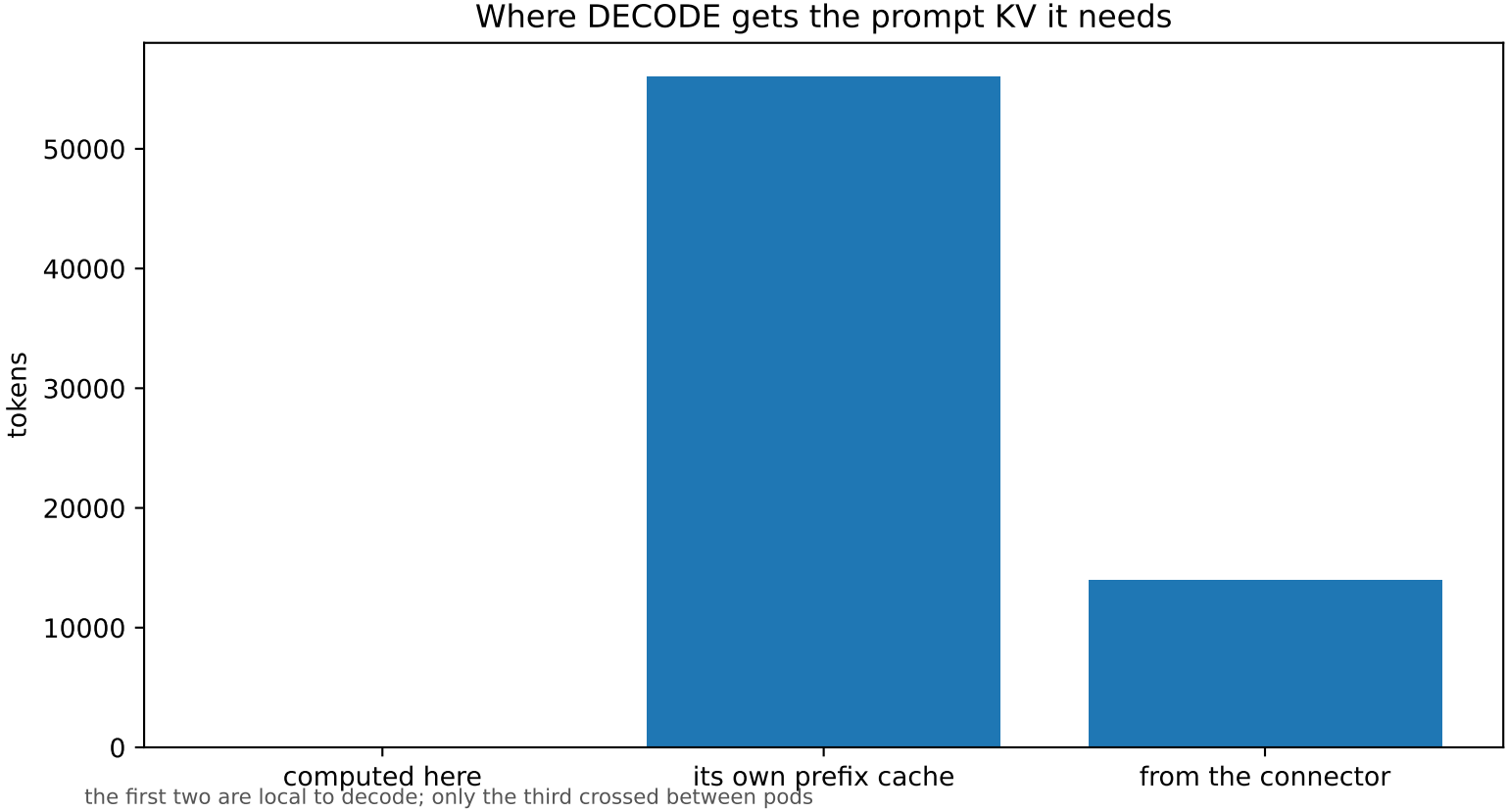
```
http://127.0.0.1:31495/d/dodona-mooncake?from=now-30m&to=now
```

gateway

```
http://127.0.0.1:31495/d/dodona-gateway?from=now-30m&to=now
```


Where a request spends its time
ms
0
10
20
30
40
50
60
70
waiting
prefill
decode
end to end
the first three are phases of the fourth


Time to first token
0.10
0.08
0.06
0.04
0.02
0.00
seconds
p50
p90
p99
dashed line is the 1.5s promise


Where DECODE gets the prompt KV it needs
tokens
0
10000
20000
30000
40000
50000
computed here
its own prefix cache
from the connector
the first two are local to decode; only the third crossed between pods